ハンズオン 07: WebViewと認証セッションの比較

この資料は、全8回の学習事項のうち7つ目「WebViewと認証セッションの比較」のハンズオンです。

ハンズオン01〜06が完了している前提で進めます。

1. この回のゴール

- ASWebAuthenticationSessionとWKWebViewの違いを理解できる
- iOS標準ブラウザ（Safari）との Cookie / セッション共有の仕組みを学べる
- 外部認証（OAuth/OpenID Connect）でのコールバックURLの役割を理解できる
- アプリ内ブラウザとシステムブラウザの使い分けを判断できる

2. 前提条件

- ハンズオン01〜06が完了している
- 以下フォルダが存在する
  - `src/WebViewHandson02/WebViewHandson02` （Xcodeプロジェクト）
- Amplify Hosting に認証機能付きWebアプリがデプロイ済み
- Xcode がインストール済み
- SwiftUIの基本的な知識

3. 進行タイムライン（目安: 90分）

- 0:00-0:10 イントロ・WebViewとASWebAuthenticationSessionの違いの解説
- 0:10-0:30 ASWebAuthenticationSessionの基本実装（Step 1）
- 0:30-0:50 コールバックURLスキームの設定（Step 2）
- 0:50-1:10 Cookie/セッション共有の検証実験（Step 3）
- 1:10-1:25 セキュリティ観点の理解と使い分け（Step 4）
- 1:25-1:30 振り返り

4. ハンズオン手順

Step 0: WKWebViewとASWebAuthenticationSessionの違いを理解する

WKWebView（既に実装済み）

特徴:

- アプリ内に埋め込まれた独立したブラウザコンポーネント
- Safariとは完全に独立した Cookie / セッションストレージを持つ
- ユーザーがSafariでログイン済みでも、WebView側では未ログイン状態になる

適している用途:

- アプリ内でWebコンテンツを表示する（ニュース記事、ヘルプページなど）

- アプリが完全にコントロールできるWeb体験を提供する
- ユーザーがアプリ外に遷移することなく、Webコンテンツを閲覧できる

セキュリティ:

- アプリ内に閉じているため、外部への遷移が制限される
- ただし、Cookieやセッションは独立しているため、SSO（Single Sign-On）には不向き

ASWebAuthenticationSession（今回実装）

特徴:

- システムレベルのブラウザセッションを使用する
- Safariと Cookie / セッション を共有できる
- ユーザーの明示的な許可が必要（セキュリティダイアログが表示される）
- OAuth 2.0 / OpenID Connect などの外部認証に最適

適している用途:

- 外部サービス（Google、Twitter、GitHubなど）でのOAuth認証
- エンタープライズSSO（企業の認証基盤との連携）
- ユーザーが既にSafariでログインしている認証情報を再利用したい場合

セキュリティ:

- システムが管理する信頼できるブラウザ環境を使用
- コールバックURLによる安全な結果の受け渡し
- アプリがCookieに直接アクセスできないため、認証情報の漏洩リスクが低い

Step 1: ASWebAuthenticationSessionの基本実装

既存のプロジェクトに、ASWebAuthenticationSessionを実装します。

1-1. 新しいSwiftファイルを作成する

1. Xcodeで `WebViewHandson02.xcodeproj` を開く
2. プロジェクトナビゲーターで `WebViewHandson02` フォルダを右クリック
3. `New File...` を選択
4. `Swift File` を選択して `Next`
5. ファイル名を `AuthenticationSessionView.swift` として `Create`

1-2. ASWebAuthenticationSessionの実装

作成した `AuthenticationSessionView.swift` に以下のコードを実装します。

シンプル版（基本機能のみ）:

```swift
import SwiftUI
import AuthenticationServices

struct AuthenticationSessionView: View {
    @State private var isAuthenticating = false
    @State private var resultMessage = ""

    // presentationContextProviderを保持するための変数
    private let presentationContext = AuthenticationPresentationContext()

    // 認証に使用するURL（外部から渡される）
    let url: URL

    // コールバックURLスキーム
    let callbackURLScheme = "myapp"

    var body: some View {
        VStack(spacing: 20) {
            Text("ASWebAuthenticationSession テスト")
                .font(.title)
                .padding()

            Button(action: {
                startAuthentication()
            }) {
                Text("認証セッションを開始")
                    .padding()
                    .background(Color.blue)
                    .foregroundColor(.white)
                    .cornerRadius(10)
            }
            .disabled(isAuthenticating)

            if !resultMessage.isEmpty {
                Text(resultMessage)
                    .padding()
                    .background(Color.gray.opacity(0.2))
                    .cornerRadius(10)
            }

            Spacer()
        }
        .padding()
    }

    private func startAuthentication() {
        isAuthenticating = true

        let session = ASWebAuthenticationSession(
            url: url,
            callbackURLScheme: callbackURLScheme
        ) { callbackURL, error in
            isAuthenticating = false

            if let error = error {
```

```
                if (error as NSError).code == ASWebAuthenticationSessionError.canceledLogin.rawValue {
                    resultMessage = "認証がキャンセルされました"
                } else {
                    resultMessage = "エラー: \(error.localizedDescription)"
                }
                return
            }

            if let callbackURL = callbackURL {
                resultMessage = "認証成功!\nコールバックURL: \(callbackURL.absoluteString)"
            }
        }

        session.presentationContextProvider = presentationContext
        session.start()
    }
}

// ASWebAuthenticationPresentationContextProvidingの実装
class AuthenticationPresentationContext: NSObject, ASWebAuthenticationPresentationContextProviding {
    func presentationAnchor(for session: ASWebAuthenticationSession) -> ASPresentationAnchor {
        if let windowScene = UIApplication.shared.connectedScenes.first as? UIWindowScene {
            return windowScene.windows.first { $0.isKeyWindow } ?? windowScene.windows.first!
        }
        fatalError("No window scene found")
    }
}

#Preview {
    AuthenticationSessionView(url: URL(string: "https://example.amplifyapp.com")!)
}
```

フル機能版（Alert対応）：

NavigationStack + Alert で結果を表示したい場合は、以下のように実装します：

```swift
import SwiftUI
import AuthenticationServices

struct AuthenticationSessionView: View {
    @State private var isAuthenticating = false
    @State private var resultMessage = ""

    private let presentationContext = AuthenticationPresentationContext()
    let url: URL
    let callbackURLScheme = "myapp"

    // コールバック用のクロージャ（オプショナル）
    var onResult: ((String, String) -> Void)?

    var body: some View {
        VStack(spacing: 20) {
            Text("ASWebAuthenticationSession テスト")
                .font(.title)
                .padding()

            Button(action: {
                startAuthentication()
            }) {
                Text("認証セッションを開始")
                    .padding()
                    .background(Color.blue)
                    .foregroundColor(.white)
                    .cornerRadius(10)
            }
            .disabled(isAuthenticating)

            if !resultMessage.isEmpty {
                ScrollView {
                    Text(resultMessage)
                        .padding()
                        .frame(maxWidth: .infinity, alignment: .leading)
                        .background(Color.gray.opacity(0.2))
                        .cornerRadius(10)
                }
            }

            Spacer()
        }
        .padding()
    }

    private func startAuthentication() {
        isAuthenticating = true

        let session = ASWebAuthenticationSession(
            url: url,
            callbackURLScheme: callbackURLScheme
        ) { callbackURL, error in
            isAuthenticating = false
```

```
            if let error = error {
                if (error as NSError).code == ASWebAuthenticationSessionError.canceledLogin.rawValue {
                    let title = "キャンセル"
                    let message = "認証がキャンセルされました"
                    resultMessage = "\(title)\n\(message)"
                    onResult?(title, message)
                } else {
                    let title = "エラー"
                    let message = error.localizedDescription
                    resultMessage = "\(title)\n\(message)"
                    onResult?(title, message)
                }
                return
            }

            if let callbackURL = callbackURL {
                let title = "認証成功"
                let message = "コールバックURL:\n\(callbackURL.absoluteString)"
                resultMessage = message
                onResult?(title, message)
            }
        }

        session.presentationContextProvider = presentationContext
        session.start()
    }
}

// ASWebAuthenticationPresentationContextProvidingの実装
class AuthenticationPresentationContext: NSObject, ASWebAuthenticationPresentationContextProviding {
    func presentationAnchor(for session: ASWebAuthenticationSession) -> ASPresentationAnchor {
        if let windowScene = UIApplication.shared.connectedScenes.first as? UIWindowScene {
            return windowScene.windows.first { $0.isKeyWindow } ?? windowScene.windows.first!
        }
        fatalError("No window scene found")
    }
}

#Preview {
    NavigationStack {
        AuthenticationSessionView(url: URL(string: "https://example.amplifyapp.com")!) { title, message in
            print("Result: \(title) - \(message)")
        }
    }
}
```

WebView と ASWebAuthenticationSession の技術的な違い:

機能	WKWebView	ASWebAuthenticationSession
JavaScript メッセージング	✅ `WKScriptMessageHandler` で直接可能	❌ 不可能（セキュリティ制約）
データの受け渡し	✅ 双方向のリアルタイム通信	⚠️ コールバックURL経由のみ
Cookie共有	❌ 独立（Safariと別）	✅ Safari と共有

機能	WKWebView	ASWebAuthenticationSession
ユースケース	アプリ内のWeb体験	OAuth/外部認証

重要な注意点:

ASWebAuthenticationSession では、WebView のような `window.webkit.messageHandlers` を使った**直接的なメッセージングはできません**。これは設計上の制約で、以下の理由があります：

1. **セキュリティ**: Safari ベースのシステムブラウザを使用するため、アプリが直接 JavaScript にアクセスできない
2. **プライバシー**: 認証情報の保護のための仕様
3. **OAuth標準**: コールバックURL を使った認証フローが標準

代わりに、**コールバックURL** を使ってデータを受け取ります：

```
myapp://callback?token=xxx&user=yyy&state=abc
```

この制約により、 `onResult` クロージャは「認証結果の表示」を親ビューに委譲するためのもので、WebView のようなリアルタイム通信ではありません。

コードの解説:

1. **状態変数の定義（8-9行目）**

   ```
   @State private var isAuthenticating = false
   @State private var resultMessage = ""
   ```

   - `isAuthenticating`：認証処理中かどうかを管理（ボタンの無効化に使用）
   - `resultMessage`：認証結果のメッセージを表示
2. **presentationContextの保持**

   ```
   private let presentationContext = AuthenticationPresentationContext()
   ```

   - `presentationContextProvider` は weak参照のプロパティのため、強参照で保持する必要がある
   - インスタンスを保持しないと、設定直後にメモリから解放されてしまう
3. **認証URLとコールバックスキームの定義（13-17行目）**

   ```
   let url: URL
   let callbackURLScheme = "myapp"
   ```

   - `url`：認証先のURL（外部から渡される）
   - `callbackURLScheme`：認証完了後にアプリに戻るためのURLスキーム
4. **ASWebAuthenticationSession の初期化**

   ```
   let session = ASWebAuthenticationSession(
       url: url,
       callbackURLScheme: callbackURLScheme
   ) { callbackURL, error in
       // 完了ハンドラー
   }
   ```

   - `url`：認証先のURL
   - `callbackURLScheme`：認証完了時に呼び出されるURLスキーム

- 第3引数: 認証完了時のクロージャ（completionHandler）

5. **completionHandler**

```
{ callbackURL, error in
    isAuthenticating = false

    if let error = error {
        // エラー処理
    }

    if let callbackURL = callbackURL {
        resultMessage = "認証成功!\nコールバックURL: \(callbackURL.absoluteString)"
    }
}
```

- 認証が完了（または失敗）したときに呼ばれる
- `callbackURL`：アプリに戻るときのURL（認証トークンなどが含まれる）
- `error`：エラーが発生した場合（ユーザーキャンセルも含む）

6. **presentationContextProvider の設定**

```
session.presentationContextProvider = presentationContext
```

- 認証画面を表示するウィンドウを指定する
- どのウィンドウ(マルチウィンドウ)上に表示するか
- iOS 13以降で必須の設定
- **重要**: `presentationContextProvider` はweak参照のため、保持されたインスタンスを使用

7. **セッションの開始（85行目）**

```
session.start()
```

- 認証セッションを実際に開始する
- ここでシステムダイアログとWebブラウザが表示される

8. **AuthenticationPresentationContext クラス（90-100行目）**

```
class AuthenticationPresentationContext: NSObject, ASWebAuthenticationPresentationContextProviding {
    func presentationAnchor(for session: ASWebAuthenticationSession) -> ASPresentationAnchor {
        if let windowScene = UIApplication.shared.connectedScenes.first as? UIWindowScene {
            return windowScene.windows.first { $0.isKeyWindow } ?? windowScene.windows.first!
        }
        fatalError("No window scene found")
    }
}
```

- 認証画面を表示するウィンドウ（ASPresentationAnchor）を返す
- iOS 15以降に対応した実装（`connectedScenes` を使用）
- 現在アクティブな `UIWindowScene` からキーウィンドウを取得

1-3. 埋め込み型WebViewの作成

WKWebViewを使った埋め込み表示用のビューを作成します。

1. Xcodeでプロジェクトナビゲーターの `WebViewHandson02` フォルダを右クリック
2. `New File...` を選択

3. `Swift File` を選択して `Next`
4. ファイル名を `EmbeddedAuthenticationView.swift` として `Create`
5. 以下のコードを実装:

```
import SwiftUI

struct EmbeddedAuthenticationView: View {
    @State private var alertTitle = ""
    @State private var alertMessage = ""
    @State private var isAlertPresented = false

    // 認証に使用するURL
    let url: URL

    var body: some View {
        VStack(spacing: 0) {
            // ヘッダー
            HStack {
                Text("WKWebView")
                    .font(.headline)
                    .padding()
                Spacer()
            }
            .background(Color(.systemGray6))

            // WebView（埋め込み表示）
            WebView(url: url) { title, message in
                alertTitle = title
                alertMessage = message
                isAlertPresented = true
            }
        }
        .alert(alertTitle, isPresented: $isAlertPresented) {
            Button("OK") {
                isAlertPresented = false
            }
        } message: {
            Text(alertMessage)
        }
    }
}

#Preview {
    EmbeddedAuthenticationView(url: URL(string: "https://example.amplifyapp.com")!)
}
```

EmbeddedAuthenticationView の特徴:

- WKWebView を内部で使用（埋め込み表示）
- ヘッダーUI付き
- JavaScript メッセージングに対応（Web → iOS 通信）
- Alert による結果表示

1-4. メイン画面にタブを追加する

`ContentView.swift` を開いて、2つのタブを追加します。

シンプルな実装（推奨）：

```
import SwiftUI

struct ContentView: View {
    private let hostedBaseURL = "https://YOUR-APP.amplifyapp.com/"

    // クエリパラメータ付きのURLを生成（iOS→Web連携）
    private var webViewURL: URL {
        var components = URLComponents(string: hostedBaseURL)!
        components.queryItems = [
            URLQueryItem(name: "from", value: "iOS"),
            URLQueryItem(name: "trainee", value: "読売新聞"),
            URLQueryItem(name: "session", value: "07")
        ]
        return components.url!
    }

    var body: some View {
        TabView {
            EmbeddedAuthenticationView(url: webViewURL)
                .tabItem {
                    Label("WKWebView", systemImage: "globe")
                }

            AuthenticationSessionView(url: URL(string: hostedBaseURL)!)
                .tabItem {
                    Label("認証セッション", systemImage: "lock.shield")
                }
        }
    }
}

#Preview {
    ContentView()
}
```

ポイント:

- `YOUR-APP` を実際のAmplify HostingのURLに置き換える
- `webViewURL` でクエリパラメータを追加（`from`、`trainee`、`session`）
- WebViewBridge.jsx の「受け取った連携情報」に表示される
- 1つ目のタブ: `EmbeddedAuthenticationView`（WKWebView ベース、埋め込み表示）
- 2つ目のタブ: `AuthenticationSessionView`（ASWebAuthenticationSession、ポップアップ表示）

EmbeddedAuthenticationView と AuthenticationSessionView の違い:

項目	EmbeddedAuthenticationView	AuthenticationSessionView
基盤技術	WKWebView	ASWebAuthenticationSession

項目	EmbeddedAuthenticationView	AuthenticationSessionView
表示方式	画面に埋め込み	ポップアップ（シート）
初期表示	最初からWebページが表示	ボタンをタップで起動
JavaScript通信	✅ 可能	❌ 不可能
Cookie共有	❌ 独立（Safariと別）	✅ Safari と共有
ユースケース	アプリ内Web体験	OAuth/外部認証

この実装により、埋め込み表示とポップアップ表示の違いを視覚的に比較できます。

（参考）フル機能実装（NavigationStack + アラート対応）

以下は、NavigationStack と Alert 機能を追加した拡張実装です。今回のハンズオンでは上記のシンプル実装で十分ですが、より高度なUI構成が必要な場合の参考にしてください。

フル機能実装（NavigationStack + アラート対応）：

WebViewBridgeの機能（Web→iOSのメッセージング）と同様に、認証結果を見やすく表示します：

```swift
import SwiftUI

struct ContentView: View {
    var body: some View {
        TabView {
            // WebViewタブ（NavigationStack付き）
            WebViewTab()
                .tabItem {
                    Label("WebView", systemImage: "globe")
                }

            // 認証セッションタブ（NavigationStack付き）
            AuthenticationSessionTab()
                .tabItem {
                    Label("認証セッション", systemImage: "lock.shield")
                }
        }
    }
}

// WebViewタブの詳細実装
struct WebViewTab: View {
    @State private var alertTitle = ""
    @State private var alertMessage = ""
    @State private var isAlertPresented = false

    // クエリパラメータ付きのURLを生成（iOS→Web連携）
    private var webViewURL: URL {
        var components = URLComponents(string: "https://YOUR-APP.amplifyapp.com")!
        components.queryItems = [
            URLQueryItem(name: "from", value: "iOS"),
            URLQueryItem(name: "trainee", value: "読売新聞"),
            URLQueryItem(name: "session", value: "07")
        ]
        return components.url!
    }

    var body: some View {
        NavigationStack {
            // onMessage クロージャで Web → iOS のメッセージを受け取る
            WebView(url: webViewURL) { title, message in
                alertTitle = title
                alertMessage = message
                isAlertPresented = true
            }
            .navigationTitle("WebView")
            .navigationBarTitleDisplayMode(.inline)
            .ignoresSafeArea(edges: .bottom)
            // UIAlertController の代わりに SwiftUI 標準の .alert を使う
            .alert(alertTitle, isPresented: $isAlertPresented) {
                Button("OK", role: .cancel) {}
            } message: {
                Text(alertMessage)
            }
        }
```

```
    }
}

// 認証セッションタブの詳細実装
struct AuthenticationSessionTab: View {
    @State private var alertTitle = ""
    @State private var alertMessage = ""
    @State private var isAlertPresented = false

    var body: some View {
        NavigationStack {
            AuthenticationSessionView(
                url: URL(string: "https://YOUR-APP.amplifyapp.com")!
            ) { title, message in
                // 認証結果をアラートで表示
                alertTitle = title
                alertMessage = message
                isAlertPresented = true
            }
            .navigationTitle("認証セッション")
            .navigationBarTitleDisplayMode(.inline)
            .alert(alertTitle, isPresented: $isAlertPresented) {
                Button("OK", role: .cancel) {}
            } message: {
                Text(alertMessage)
            }
        }
    }
}

#Preview {
    ContentView()
}
```

AuthenticationSessionView の更新版:

```swift
import SwiftUI
import AuthenticationServices

struct AuthenticationSessionView: View {
    @State private var isAuthenticating = false
    @State private var resultMessage = ""

    private let presentationContext = AuthenticationPresentationContext()
    let url: URL
    let callbackURLScheme = "myapp"

    // コールバック用のクロージャ（オプショナル）
    var onResult: ((String, String) -> Void)?

    var body: some View {
        VStack(spacing: 20) {
            Text("ASWebAuthenticationSession テスト")
                .font(.title)
                .padding()

            Button(action: {
                startAuthentication()
            }) {
                Text("認証セッションを開始")
                    .padding()
                    .background(Color.blue)
                    .foregroundColor(.white)
                    .cornerRadius(10)
            }
            .disabled(isAuthenticating)

            if !resultMessage.isEmpty {
                ScrollView {
                    Text(resultMessage)
                        .padding()
                        .frame(maxWidth: .infinity, alignment: .leading)
                        .background(Color.gray.opacity(0.2))
                        .cornerRadius(10)
                }
            }

            Spacer()
        }
        .padding()
    }

    private func startAuthentication() {
        isAuthenticating = true

        let session = ASWebAuthenticationSession(
            url: url,
            callbackURLScheme: callbackURLScheme
        ) { callbackURL, error in
            isAuthenticating = false
```

```
            if let error = error {
                if (error as NSError).code == ASWebAuthenticationSessionError.canceledLogin.rawValue {
                    let title = "キャンセル"
                    let message = "認証がキャンセルされました"
                    resultMessage = "\(title)\n\(message)"
                    onResult?(title, message)
                } else {
                    let title = "エラー"
                    let message = error.localizedDescription
                    resultMessage = "\(title)\n\(message)"
                    onResult?(title, message)
                }
                return
            }

            if let callbackURL = callbackURL {
                let title = "認証成功"
                let message = "コールバックURL:\n\(callbackURL.absoluteString)"
                resultMessage = message
                onResult?(title, message)
            }
        }

        session.presentationContextProvider = presentationContext
        session.start()
    }
}

// ASWebAuthenticationPresentationContextProvidingの実装
class AuthenticationPresentationContext: NSObject, ASWebAuthenticationPresentationContextProviding {
    func presentationAnchor(for session: ASWebAuthenticationSession) -> ASPresentationAnchor {
        if let windowScene = UIApplication.shared.connectedScenes.first as? UIWindowScene {
            return windowScene.windows.first { $0.isKeyWindow } ?? windowScene.windows.first!
        }
        fatalError("No window scene found")
    }
}

#Preview {
    AuthenticationSessionView(url: URL(string: "https://example.amplifyapp.com")!)
}
```

技術的な違いの説明:

機能	WKWebView	ASWebAuthenticationSession
JavaScript メッセージング	✅ `WKScriptMessageHandler` で直接可能	❌ 不可能
データの受け渡し	✅ 双方向のリアルタイム通信	⚠️ コールバックURL経由のみ
ユースケース	アプリ内のWeb体験	OAuth/外部認証

重要な注意点:

ASWebAuthenticationSession では、WebView のような `window.webkit.messageHandlers` を使った直接的なメッセージングは**できません**。これは設計上の制約で、セキュリティのためです。

- Safari ベースのシステムブラウザを使用
- アプリが直接 JavaScript にアクセスできない
- 認証情報の保護のための仕様

代わりに、**コールバックURL** を使ってデータを受け取ります：

myapp://callback?token=xxx&user=yyy
実装の選択:

今回のハンズオンで実装した構成：

要素	実装内容	説明
WebViewタブ	EmbeddedAuthenticationView	WKWebViewベース、埋め込み表示、Alert対応
認証セッションタブ	AuthenticationSessionView	ASWebAuthenticationSession、ポップアップ表示

EmbeddedAuthenticationView の利点:
- WebView コンポーネントを再利用
- ヘッダーUI付きで見やすい
- JavaScript メッセージング（`window.webkit.messageHandlers`）に対応
- Alert による結果表示機能

比較のポイント:
- 主目的は「埋め込み表示（WKWebView）とポップアップ表示（ASWebAuthenticationSession）の比較」
- Cookie/セッション共有の違いを検証
- 表示方式と認証フローの違いを体験

（参考）さらにカスタマイズが必要な場合:
上記の「フル機能実装」セクションを参照してください。NavigationStack による複雑なナビゲーション構造が必要な場合に有用です。

Step 2: カスタムURLスキームの設定

ASWebAuthenticationSessionがコールバックURLを受け取れるように、アプリにカスタムURLスキームを設定します。

2-1. URL Schemesの設定

1. Xcodeでプロジェクトを選択（一番上の `WebViewHandson02`）
2. `TARGETS` > `WebViewHandson02` を選択
3. `Info` タブを開く
4. `URL Types` セクションを展開（なければ `+` ボタンで追加）
5. 以下の値を設定:
   - **Identifier**: `com.example.WebViewHandson02`（アプリのBundle IDと同じでOK）
   - **URL Schemes**: `myapp`
   - **Role**: `Editor`



2-2. コールバックURL処理の実装（iOS 14+）

`WebViewHandson02App.swift` を開いて、コールバックURLを処理する準備をします。

```swift
import SwiftUI

@main
struct WebViewHandson02App: App {
    var body: some Scene {
        WindowGroup {
            ContentView()
```

```
                .onOpenURL { url in
                    print("Received callback URL: \(url)")
                    // ここで認証トークンなどを処理できる
                }
        }
    }
}
```

注意:

- 今回のハンズオンでは、実際に外部認証サービスと連携していないため、コールバックURLは返ってきません
- しかし、将来的にOAuth認証を実装する際に、この設定が必要になります

Step 3: Cookie/セッション共有の検証実験

WKWebViewとASWebAuthenticationSessionで、Cookie/セッションの共有がどのように異なるかを実験します。

実験の流れ

1. **Safariでログイン**
2. **WKWebViewで確認** → ログイン状態が共有されていないことを確認
3. **ASWebAuthenticationSessionで確認** → ログイン状態が共有されていることを確認

3-1. Safariでログインする

1. iPhoneのSafariアプリを開く
2. Amplify HostingのURLにアクセス（ `https://YOUR-APP.amplifyapp.com` ）
3. サインインする（ハンズオン06で作成したアカウント）
4. ログイン状態を確認する（Todoが表示される）

3-2. WKWebViewでログイン状態を確認する

1. Xcodeで `WebViewHandson02` アプリを実行
2. `WKWebView` タブを開く（EmbeddedAuthenticationView が表示される）
3. **結果:** ログインしていない状態（サインイン画面が表示される）

理由:

- EmbeddedAuthenticationView は内部で WKWebView を使用
- WKWebView は独立したブラウザコンテキストを持つ
- Safariとは別のCookie / セッション ストレージを使用している

3-3. ASWebAuthenticationSessionでログイン状態を確認する

1. `認証セッション` タブを開く
2. `認証セッションを開始` ボタンをタップ
3. システムダイアログが表示される → `Continue` をタップ
4. **結果:** Safariでログイン済みなので、既にログイン状態になっている（Todoが表示される）

理由:

- ASWebAuthenticationSessionはシステムレベルのブラウザを使用

- Safariと同じ Cookie / セッション を共有している

3-4. 実験結果のまとめ

ブラウザ種別	表示方式	Safariとのセッション共有	ユーザー許可	用途
EmbeddedAuthenticationView（WKWebView）	埋め込み	× 共有されない	不要	アプリ内コンテンツ表示
AuthenticationSessionView（ASWebAuthenticationSession）	ポップアップ	○ 共有される	必要	外部認証（OAuth等）

重要なポイント:

- **EmbeddedAuthenticationView** は WKWebView を内部で使用しているため、Safari とは独立した Cookie/セッションを持つ
- 埋め込み表示が必要で、かつ Safari とのセッション共有が不要な場合は WKWebView を使用
- OAuth/OpenID Connect などの外部認証で Safari とのセッション共有が必要な場合は ASWebAuthenticationSession を使用

Step 4: セキュリティ観点の理解と使い分け

4-1. ASWebAuthenticationSessionのセキュリティメリット

1. **ユーザーの明示的な許可が必要**
   - システムダイアログが表示され、ユーザーが意図的に許可する
   - フィッシング攻撃のリスクを軽減
2. **アプリがCookieに直接アクセスできない**
   - 認証情報はシステムが管理
   - アプリは結果（コールバックURL）のみを受け取る
   - 認証トークンが安全に保護される
3. **システムが信頼できるブラウザ環境を提供**
   - Safariのセキュリティメカニズムがそのまま適用される
   - SSL/TLS証明書の検証、セキュアな通信

4-2. コールバックURLの役割

OAuth 2.0などの外部認証では、以下のフローで認証が行われます:

```
1. アプリ → 認証サーバー: 認証リクエスト
   例: https://auth.example.com/login?client_id=xxx&redirect_uri=myapp://callback

2. ユーザー → 認証サーバー: ログイン（ID/パスワード入力）

3. 認証サーバー → アプリ: 認証結果をコールバックURLで返す
   例: myapp://callback?code=abc123&state=xyz

4. アプリ: 認証コードを使ってアクセストークンを取得
```

コールバックURLのセキュリティ:

- アプリ固有のURLスキームを使用（ `myapp://` ）
- 他のアプリが勝手に受け取れないようにする

- `state` パラメータでCSRF攻撃を防ぐ

4-3. 使い分けのガイドライン

シナリオ	推奨	理由
ニュース記事の表示	WKWebView	アプリ内で完結、認証不要
ヘルプページの表示	WKWebView	アプリ内で完結、認証不要
Google OAuth認証	ASWebAuthenticationSession	外部認証、セッション共有が必要
企業SSOログイン	ASWebAuthenticationSession	外部認証、セキュリティが重要
自社アプリの認証画面	WKWebView または Amplify UI	アプリ内で管理できる

4-4. Amplify認証との連携（今後の展開）

今回のハンズオンでは、Amplify UIライブラリを使って認証を実装しました。
しかし、以下のようなケースでは ASWebAuthenticationSession が有効です:

1. **Cognitoのホステッド UIを使用する場合**
   - Cognito User Poolの認証画面をWebで表示
   - Googleなどのソーシャルログインを使用
2. **外部IDプロバイダー（IdP）と連携する場合**
   - SAML認証（企業の認証基盤）
   - カスタムOAuth実装
3. **Webとネイティブアプリでセッションを共有したい場合**
   - ユーザーがWebでログイン済みなら、アプリでも自動ログイン

Step 5: 動作確認とまとめ

5-1. 最終動作確認

1. アプリを再実行
2. `WKWebView` タブ（EmbeddedAuthenticationView）と **`認証セッション`** タブを切り替える
3. それぞれの表示方式の違いを確認:
   - **WKWebView タブ**: 埋め込み表示、最初からWebページが表示
   - **認証セッション タブ**: ボタンをタップするとポップアップ表示
4. ログイン状態の違いを確認:
   - **WKWebView タブ**: Safariと独立したセッション（未ログイン）
   - **認証セッション タブ**: Safariとセッション共有（Safariでログイン済みならログイン状態）
5. Safariでログアウトして、再度 **`認証セッション`** で確認 → ログアウト状態になる

確認ポイント:

- 埋め込み表示（WKWebView）とポップアップ表示（ASWebAuthenticationSession）の UI の違い
- Cookie/セッション共有の有無による動作の違い

5-2. この回で学んだこと

- ✅ WKWebViewとASWebAuthenticationSessionの違い

- ✅ Cookie / セッション の共有メカニズム
- ✅ カスタムURLスキームの設定とコールバックURL処理
- ✅ 外部認証（OAuth）のセキュリティ理解
- ✅ アプリ内ブラウザとシステムブラウザの使い分け

5-3. 次回に向けて

次回（第8回）では、以下を学習します:

- GraphQLスキーマの `@auth(owner)` ルール
- ユーザー単位のデータ保護
- SwiftネイティブからのAmplify API呼び出し
- ReactとSwiftの両方からの同一データ操作

5. トラブルシューティング

問題1: ASWebAuthenticationSessionが起動しない

原因:

- URLが不正
- `presentationContextProvider` が設定されていない

解決策:

- URLを確認（`https://` で始まる正しいURL）
- `AuthenticationPresentationContext` クラスが実装されているか確認

問題2: コールバックURLが受け取れない

原因:

- URL Schemesが設定されていない
- URLスキームが一致していない

解決策:

- Xcode の `Info` > `URL Types` で `myapp` が設定されているか確認
- `callbackURLScheme` の値と一致しているか確認

問題3: 「Instance will be immediately deallocated」の警告が出る

原因:

```
// ❌ 悪い例
session.presentationContextProvider = AuthenticationPresentationContext()
```

- `presentationContextProvider` は weak参照のプロパティ
- 新しく作成したインスタンスがどこからも強参照されていないため、すぐに解放される

解決策:

```
// ✅ 良い例：プロパティとして保持
struct AuthenticationSessionView: View {
    private let presentationContext = AuthenticationPresentationContext()

    // ...

    func startAuthentication() {
        session.presentationContextProvider = presentationContext
    }
}
```

- インスタンスをプロパティとして保持する
- Viewのライフサイクルに合わせて保持される

問題4: 「このサイトへのサインインを許可しますか?」ダイアログが表示されない

原因:

- iOS 13以降では、ユーザーの許可が必要
- すでに許可済みの場合、再度は表示されない

解決策:

- 初回起動時には必ず表示される
- 動作に問題なければ、そのまま進めてOK

問題5: 「'windows' was deprecated in iOS 15.0」の警告が出る

原因:

```
// ❌ iOS 15以降では非推奨
return UIApplication.shared.windows.first { $0.isKeyWindow } ?? UIWindow()
```

- `UIApplication.shared.windows` は iOS 15で非推奨
- `UIWindow()` イニシャライザは iOS 26で非推奨
- 代わりに `UIWindowScene` を使用する必要がある

解決策:

```
// ✅ iOS 15以降に対応した実装
func presentationAnchor(for session: ASWebAuthenticationSession) -> ASPresentationAnchor {
    if let windowScene = UIApplication.shared.connectedScenes.first as? UIWindowScene {
        return windowScene.windows.first { $0.isKeyWindow } ?? windowScene.windows.first!
    }
    fatalError("No window scene found")
}
```

- `connectedScenes` から `UIWindowScene` を取得
- そのシーンの `windows` からキーウィンドウを返す
- マルチウィンドウ環境にも対応した実装

問題6: Safariのセッションが共有されない

原因:

- プライベートブラウズモードになっている
- シミュレーターのCookieが無効化されている

解決策:

- Safariでプライベートブラウズをオフにする
- シミュレーターを再起動してから再度試す

6. 参考資料

- Apple Developer - ASWebAuthenticationSession
- OAuth 2.0 の仕組みと認証フロー
- AWS Amplify - Social Sign-in
- WKWebView vs SFSafariViewController vs ASWebAuthenticationSession